

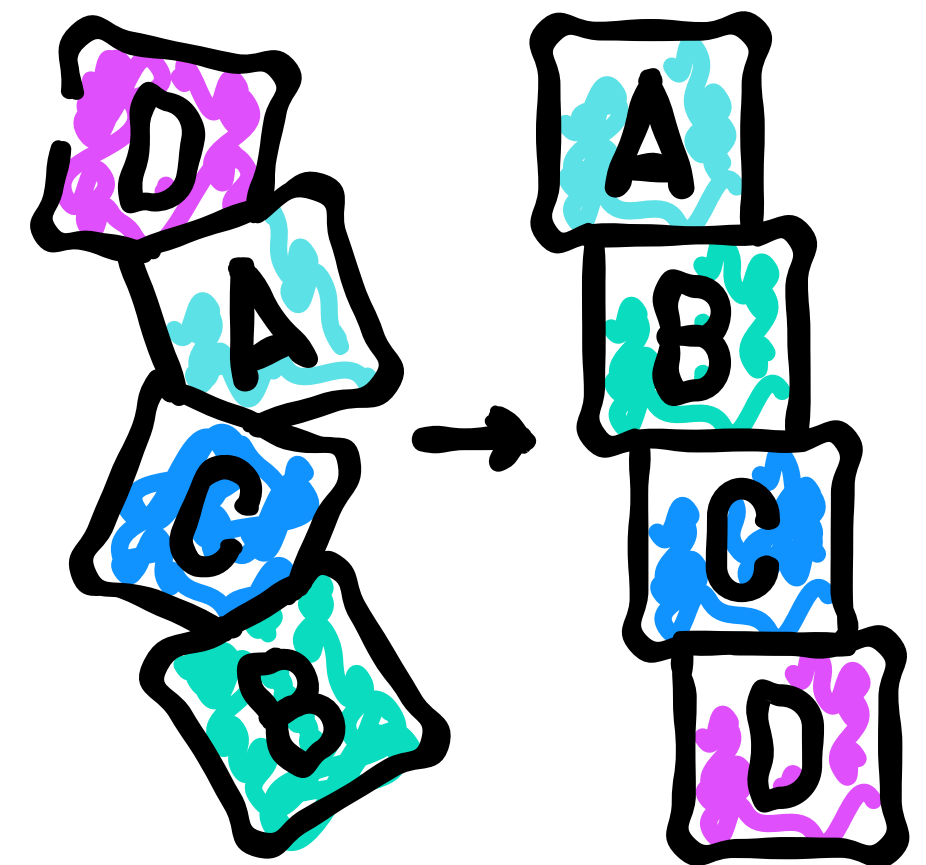
MAIO | 2026

ANÁLISE DE ALGORITMOS

shell sort

LUIZ FELIPE MIRANDA
STEFANY FIGUEIREDO

O Shell Sort é um algoritmo de ordenação proposto por Donald Shell em 1959, com o objetivo de melhorar o desempenho do Insertion Sort



Exemplo de Funcionamento do Shell Sort

Vetor Inicial:

[8, 5, 3, 7, 6, 2, 1, 4]

- Tamanho do vetor: 8
- Gap inicial:

$$\text{gap} = 8 / 2 = 4$$

Exemplo de Funcionamento do Shell Sort

Comparações realizadas:

(0,4) $\rightarrow$ 8 e 6

(1,5) $\rightarrow$ 5 e 2

(2,6) $\rightarrow$ 3 e 1

(3,7) $\rightarrow$ 7 e 4

Trocas:

8 > 6 $\rightarrow$ [6, 5, 3, 7, 8, 2, 1, 4]

5 > 2 $\rightarrow$ [6, 2, 3, 7, 8, 5, 1, 4]

3 > 1 $\rightarrow$ [6, 2, 1, 7, 8, 5, 3, 4]

7 > 4 $\rightarrow$ [6, 2, 1, 4, 8, 5, 3, 7]

Resultado da Iteração:

[6, 2, 1, 4, 8, 5, 3, 7]

Exemplo de Funcionamento do Shell Sort

Segunda Iteração (gap = 2)

Divisão em grupos

- Índices pares

[6, 1, 8, 3]

- Índices ímpares

[2, 4, 5, 7]

Exemplo de Funcionamento do Shell Sort

Grupo 1:

$6 > 1 \rightarrow [1, 6, 8, 3]$

$8 > 3 \rightarrow [1, 6, 3, 8]$

$6 > 3 \rightarrow [1, 3, 6, 8]$

Grupo 2:

Já estava ordenado

Vetor atualizado:

$[1, 2, 3, 4, 6, 5, 8, 7]$

Exemplo de Funcionamento do Shell Sort

Terceira Iteração (gap = 1)

Com gap = 1, o Shell Sort passa a funcionar como o Insertion Sort.

Comparações finais:

$6 > 5 \rightarrow [1, 2, 3, 4, 5, 6, 8, 7]$

$8 > 7 \rightarrow [1, 2, 3, 4, 5, 6, 7, 8]$

Vetor Ordenado:

$[1, 2, 3, 4, 5, 6, 7, 8]$

Análise do Exemplo

- O algoritmo realiza a ordenação de forma gradual
- Grandes deslocamentos acontecem primeiro
- Ajustes menores acontecem nas etapas finais
- Elementos distantes são reposicionados rapidamente
- O vetor fica cada vez mais organizado
- Quando $\text{gap} = 1$, o vetor já está quase ordenado
- Isso torna a etapa final muito mais eficiente

Análise do Algoritmo

Implementação em Linguagem C

```
VOID SHELLSORT(INT VETOR[], INT N) {  
    INT GAP, I, J, TEMP;  
  
    FOR (GAP = N / 2; GAP > 0; GAP /= 2) {  
  
        FOR (I = GAP; I < N; I++) {  
            TEMP = VETOR[I];  
  
            FOR (J = I; J >= GAP && VETOR[J - GAP] > TEMP; J -= GAP) {  
                VETOR[J] = VETOR[J - GAP];  
            }  
  
            VETOR[J] = TEMP;  
        }  
    }  
}
```

Análise de Desempenho

Comportamento do Shell Sort:

- O algoritmo reduz a desordem gradualmente
- Grandes movimentações ocorrem no início
- Pequenos ajustes acontecem no final
- O vetor chega quase ordenado na última etapa